In [1]:
```
import pandas as pd

# df1 = Transactions (The "Sales" data)
df1 = pd.read_excel('QVI_transaction_data.xlsx')

# df2 = Customers (The "Behavior" data)
df2 = pd.read_csv('QVI_purchase_behaviour.csv')
```

In [5]:
```
df1.info()
df2.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 264836 entries, 0 to 264835
Data columns (total 8 columns):
 #   Column          Non-Null Count   Dtype  
---  ------          --------------   -----  
 0   DATE            264836 non-null  int64  
 1   STORE_NBR       264836 non-null  int64  
 2   LYLTY_CARD_NBR  264836 non-null  int64  
 3   TXN_ID          264836 non-null  int64  
 4   PROD_NBR        264836 non-null  int64  
 5   PROD_NAME       264836 non-null  str    
 6   PROD_QTY        264836 non-null  int64  
 7   TOT_SALES       264836 non-null  float64
dtypes: float64(1), int64(6), str(1)
memory usage: 16.2 MB
<class 'pandas.DataFrame'>
RangeIndex: 72637 entries, 0 to 72636
Data columns (total 3 columns):
 #   Column            Non-Null Count  Dtype 
---  ------            --------------  ----- 
 0   LYLTY_CARD_NBR    72637 non-null  int64 
 1   LIFESTAGE         72637 non-null  str   
 2   PREMIUM_CUSTOMER  72637 non-null  str   
dtypes: int64(1), str(2)
memory usage: 1.7 MB
```

In [3]:
```
df1.head()
```

Out[3]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PRO
0	43390	1	1000	1	5	Natural Chip Compny SeaSalt175g	
1	43599	1	1307	348	66	CCs Nacho Cheese 175g	
2	43605	1	1343	383	61	Smiths Crinkle Cut Chips Chicken 170g	
3	43329	2	2373	974	69	Smiths Chip Thinly S/Cream&Onion 175g	
4	43330	2	2426	1038	108	Kettle Tortilla ChpsHny&Jlpno Chili 150g	

In [4]:
```
df2.head()
```

Out[4]:

	LYLTY_CARD_NBR	LIFESTAGE	PREMIUM_CUSTOMER
0	1000	YOUNG SINGLES/COUPLES	Premium
1	1002	YOUNG SINGLES/COUPLES	Mainstream
2	1003	YOUNG FAMILIES	Budget
3	1004	OLDER SINGLES/COUPLES	Mainstream
4	1005	MIDAGE SINGLES/COUPLES	Mainstream

In [6]:
```
# Convert integer dates to actual datetime objects
df1['DATE'] = pd.to_datetime(df1['DATE'], unit='D', origin='1899-12-30')

# Run this to verify the first few dates look normal now
df1['DATE'].head()
```

Out[6]:
```
0   2018-10-17
1   2019-05-14
2   2019-05-20
3   2018-08-17
4   2018-08-18
Name: DATE, dtype: datetime64[s]
```

In [7]:
```
# Look at unique product names to see if 'salsa' is in there
# This prints every unique product name in the dataset
print(df1['PROD_NAME'].unique())
```

```
<StringArray>
[  'Natural Chip        Compny SeaSalt175g',
                'CCs Nacho Cheese    175g',
   'Smiths Crinkle Cut  Chips Chicken 170g',
   'Smiths Chip Thinly  S/Cream&Onion 175g',
 'Kettle Tortilla ChpsHny&Jlpno Chili 150g',
 'Old El Paso Salsa   Dip Tomato Mild 300g',
 'Smiths Crinkle Chips Salt & Vinegar 330g',
    'Grain Waves         Sweet Chilli 210g',
  'Doritos Corn Chip Mexican Jalapeno 150g',
    'Grain Waves Sour    Cream&Chives 210G',
 ...
         'Doritos Cheese      Supreme 330g',
      'Smiths Crinkle Cut  Snag&Sauce 150g',
   'WW Sour Cream &OnionStacked Chips 160g',
                'RRD Lime & Pepper   165g',
   'Natural ChipCo Sea  Salt & Vinegr 175g',
    'Red Rock Deli Chikn&Garlic Aioli 150g',
      'RRD SR Slow Rst     Pork Belly 150g',
                'RRD Pc Sea Salt     165g',
       'Smith Crinkle Cut   Bolognese 150g',
                'Doritos Salsa Mild  300g']
Length: 114, dtype: str
```

In [8]:
```
# This creates a temporary view of only the salsa products
salsa_products = df1[df1['PROD_NAME'].str.contains("salsa", case=False)]
print(f"Number of salsa transactions found: {len(salsa_products)}")
print(salsa_products['PROD_NAME'].unique())
```

```
Number of salsa transactions found: 18094
<StringArray>
['Old El Paso Salsa   Dip Tomato Mild 300g',
 'Red Rock Deli SR    Salsa & Mzzrlla 150g',
    'Smiths Crinkle Cut  Tomato Salsa 150g',
          'Doritos Salsa       Medium 300g',
 'Old El Paso Salsa   Dip Chnky Tom Ht300g',
           'Woolworths Mild     Salsa 300g',
  'Old El Paso Salsa   Dip Tomato Med 300g',
           'Woolworths Medium   Salsa 300g',
                'Doritos Salsa Mild  300g']
Length: 9, dtype: str
```

In [9]:
```
# The ~ symbol means "NOT"
df1 = df1[~df1['PROD_NAME'].str.lower().str.contains("salsa")]
```

In [10]:
```
print(df1['PROD_NAME'].unique())
```

```
<StringArray>
[  'Natural Chip        Compny SeaSalt175g',
                'CCs Nacho Cheese    175g',
   'Smiths Crinkle Cut  Chips Chicken 170g',
   'Smiths Chip Thinly  S/Cream&Onion 175g',
 'Kettle Tortilla ChpsHny&Jlpno Chili 150g',
 'Smiths Crinkle Chips Salt & Vinegar 330g',
    'Grain Waves         Sweet Chilli 210g',
  'Doritos Corn Chip Mexican Jalapeno 150g',
    'Grain Waves Sour    Cream&Chives 210G',
    'Kettle Sensations   Siracha Lime 150g',
 ...
                'RRD Salt & Vinegar  165g',
         'Doritos Cheese      Supreme 330g',
      'Smiths Crinkle Cut  Snag&Sauce 150g',
   'WW Sour Cream &OnionStacked Chips 160g',
                'RRD Lime & Pepper   165g',
   'Natural ChipCo Sea  Salt & Vinegr 175g',
    'Red Rock Deli Chikn&Garlic Aioli 150g',
      'RRD SR Slow Rst     Pork Belly 150g',
                'RRD Pc Sea Salt     165g',
       'Smith Crinkle Cut   Bolognese 150g']
Length: 105, dtype: str
```

In [11]:
```
df1.describe()
```

Out[11]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_C
count	246742	246742.000000	2.467420e+05	2.467420e+05	246742.000000	246742.000
mean	2018-12-30 01:19:01	135.051098	1.355310e+05	1.351311e+05	56.351789	1.908
min	2018-07-01 00:00:00	1.000000	1.000000e+03	1.000000e+00	1.000000	1.000
25%	2018-09-30 00:00:00	70.000000	7.001500e+04	6.756925e+04	26.000000	2.000
50%	2018-12-30 00:00:00	130.000000	1.303670e+05	1.351830e+05	53.000000	2.000
75%	2019-03-31 00:00:00	203.000000	2.030840e+05	2.026538e+05	87.000000	2.000
max	2019-06-30 00:00:00	272.000000	2.373711e+06	2.415841e+06	114.000000	200.000
std	NaN	76.787096	8.071528e+04	7.814772e+04	33.695428	0.659

In [12]:
```
# Show rows where someone bought more than 10 packets at once
outliers = df1[df1['PROD_QTY'] > 10]
outliers
```

Out[12]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QT
69762	2018-08-19	226	226000	226201	4	Dorito Corn Chp Supreme 380g	2(
69763	2019-05-20	226	226000	226210	4	Dorito Corn Chp Supreme 380g	2(

In [13]:
```
# Filter out the wholesale customer
df1 = df1[df1['LYLTY_CARD_NBR'] != 226000]

# Double check that the max quantity is now reasonable
print("New max quantity:", df1['PROD_QTY'].max())
```

```
New max quantity: 5
```

In [15]:
```
# Extract digits from the PROD_NAME column
df1['PACK_SIZE'] = df1['PROD_NAME'].str.extract(r'(\d+)').astype(float)

# Check the unique pack sizes to make sure it worked
print(df1['PACK_SIZE'].unique())
```

```
[175. 170. 150. 330. 210. 270. 220. 125. 110. 134. 380. 180. 165. 135.
 250. 200. 160. 190.  90.  70.]
```

In [16]:
```
# Create a BRAND column by taking the first word of PROD_NAME
df1['BRAND'] = df1['PROD_NAME'].str.split().str[0]

# Look at the unique brands to find the "doubles"
print(df1['BRAND'].unique())
```

```
['Natural' 'CCs' 'Smiths' 'Kettle' 'Grain' 'Doritos' 'Twisties' 'WW'
 'Thins' 'Burger' 'NCC' 'Cheezels' 'Infzns' 'Red' 'Pringles' 'Dorito'
 'Infuzions' 'Smith' 'GrnWves' 'Tyrrells' 'Cobs' 'French' 'RRD' 'Tostitos'
 'Cheetos' 'Woolworths' 'Snbts' 'Sunbites']
```

In [17]:
```
# Create a dictionary to map the "messy" names to a single "clean" name
brand_map = {
    "Red": "RRD",
    "Dorito": "Doritos",
    "Infzns": "Infuzions",
    "Smith": "Smiths",
    "Snbts": "Sunbites",
    "WW": "Woolworths",
    "Natural": "NCC",
    "Grain": "GrnWves"
}
```

```
# Apply the mapping to your BRAND column
df1['BRAND'] = df1['BRAND'].replace(brand_map)

# Check the unique values again to ensure they are merged
print(df1['BRAND'].unique())
```

```
['NCC' 'CCs' 'Smiths' 'Kettle' 'GrnWves' 'Doritos' 'Twisties' 'Woolworths'
 'Thins' 'Burger' 'Cheezels' 'Infuzions' 'RRD' 'Pringles' 'Tyrrells'
 'Cobs' 'French' 'Tostitos' 'Cheetos' 'Sunbites']
```

In [18]:
```
# Check if any rows are missing information
print(df2.isnull().sum())
```

```
LYLTY_CARD_NBR      0
LIFESTAGE           0
PREMIUM_CUSTOMER    0
dtype: int64
```

In [19]:
```
# Check if loyalty cards are unique
print("Total rows:", len(df2))
print("Unique cards:", df2['LYLTY_CARD_NBR'].nunique())

# If those two numbers aren't the same, you have duplicates!
```

```
Total rows: 72637
Unique cards: 72637
```

In [20]:
```
# See the different types of customers
print(df2['LIFESTAGE'].unique())
print(df2['PREMIUM_CUSTOMER'].unique())
```

```
<StringArray>
[ 'YOUNG SINGLES/COUPLES',         'YOUNG FAMILIES',  'OLDER SINGLES/COUPLES',
 'MIDAGE SINGLES/COUPLES',           'NEW FAMILIES',         'OLDER FAMILIES',
               'RETIREES']
Length: 7, dtype: str
<StringArray>
['Premium', 'Mainstream', 'Budget']
Length: 3, dtype: str
```

In [21]:
```
# Merge transactions (df1) with customer behavior (df2)
# We use a 'left' join to keep all transaction records
data = pd.merge(df1, df2, on='LYLTY_CARD_NBR', how='left')

# Check for any missing values after the merge
print(data.isnull().sum())
```

```
DATE                0
STORE_NBR           0
LYLTY_CARD_NBR      0
TXN_ID              0
PROD_NBR            0
PROD_NAME           0
PROD_QTY            0
TOT_SALES           0
PACK_SIZE           0
BRAND               0
LIFESTAGE           0
PREMIUM_CUSTOMER    0
dtype: int64
```

In [22]:
```
# Summary of total sales, number of customers, and average price per unit
summary = data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER']).agg(
    Total_Sales=('TOT_SALES', 'sum'),
    Unique_Customers=('LYLTY_CARD_NBR', 'nunique'),
    Average_Price_Per_Unit=('TOT_SALES', lambda x: x.sum() / data.loc[x.index, 'PRO
).reset_index()

print(summary.sort_values(by='Total_Sales', ascending=False))
```

```
                LIFESTAGE PREMIUM_CUSTOMER  Total_Sales  Unique_Customers  \
6          OLDER FAMILIES           Budget    156863.75              4611
19   YOUNG SINGLES/COUPLES       Mainstream    147582.20              7917
13               RETIREES       Mainstream    145168.95              6358
15         YOUNG FAMILIES           Budget    129717.95              3953
9    OLDER SINGLES/COUPLES           Budget    127833.60              4849
10   OLDER SINGLES/COUPLES       Mainstream    124648.50              4858
11   OLDER SINGLES/COUPLES          Premium    123537.55              4682
12               RETIREES           Budget    105916.30              4385
7          OLDER FAMILIES       Mainstream     96413.55              2788
14               RETIREES          Premium     91296.65              3812
16         YOUNG FAMILIES       Mainstream     86338.25              2685
1   MIDAGE SINGLES/COUPLES       Mainstream     84734.25              3298
17         YOUNG FAMILIES          Premium     78571.70              2398
8          OLDER FAMILIES          Premium     75242.60              2231
18   YOUNG SINGLES/COUPLES           Budget     57122.10              3647
2   MIDAGE SINGLES/COUPLES          Premium     54443.85              2369
20   YOUNG SINGLES/COUPLES          Premium     39052.30              2480
0   MIDAGE SINGLES/COUPLES           Budget     33345.70              1474
3             NEW FAMILIES           Budget     20607.45              1087
4             NEW FAMILIES       Mainstream     15979.70               830
5             NEW FAMILIES          Premium     10760.80               575

    Average_Price_Per_Unit
6                 3.747969
19                4.074043
13                3.852986
15                3.761903
9                 3.887529
10                3.822753
11                3.897698
12                3.932731
7                 3.736380
14                3.924037
16                3.722439
1                 3.994449
17                3.759232
8                 3.717703
18                3.685297
2                 3.780823
20                3.692889
0                 3.753878
3                 3.931969
4                 3.935887
5                 3.886168
```

In [23]:
```python
# Filter for the target segment
target_segment = data[(data['LIFESTAGE'] == 'YOUNG SINGLES/COUPLES') &
                      (data['PREMIUM_CUSTOMER'] == 'Mainstream')]

# See their top 5 brands
top_brands = target_segment['BRAND'].value_counts(normalize=True).head(5)
print("Top Brands for Mainstream Young Singles/Couples:")
print(top_brands)
```

```
Top Brands for Mainstream Young Singles/Couples:
BRAND
Kettle       0.196684
Doritos      0.121725
Pringles     0.118451
Smiths       0.098291
Infuzions    0.063958
Name: proportion, dtype: float64
```

In [24]:
```python
# Check pack size preferences for Mainstream Young Singles/Couples
pack_size_pref = target_segment['PACK_SIZE'].value_counts(normalize=True).head(5)

print("Top Pack Sizes for Mainstream Young Singles/Couples:")
print(pack_size_pref)
```

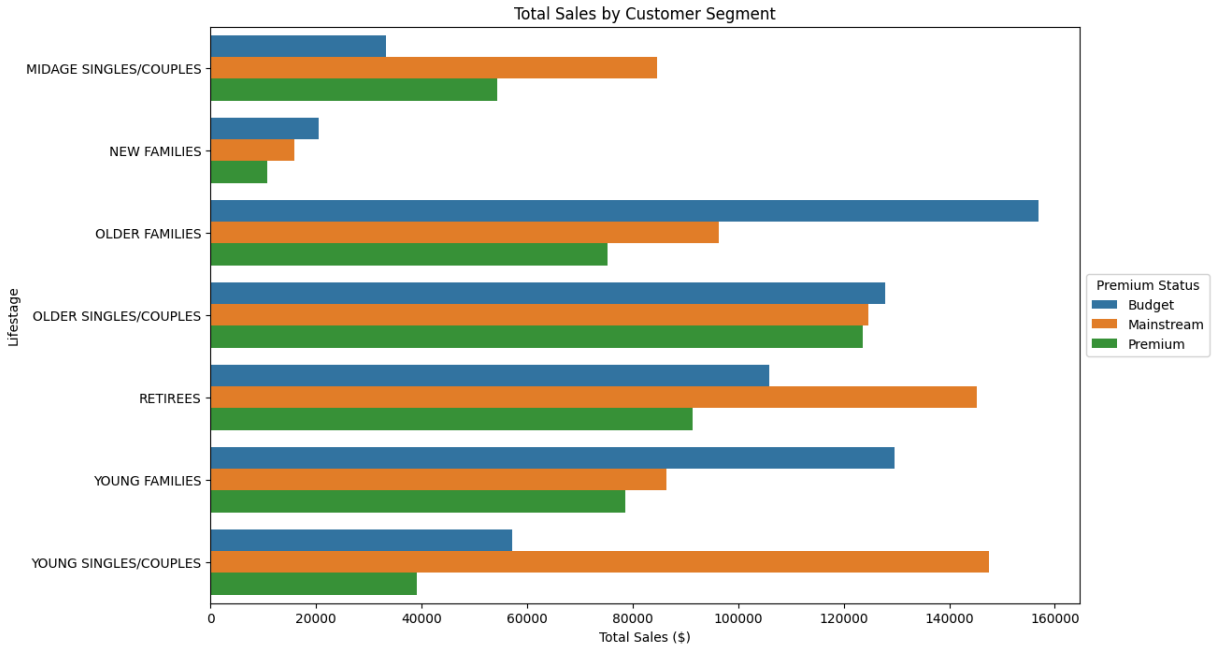
```
Top Pack Sizes for Mainstream Young Singles/Couples:
PACK_SIZE
175.0    0.255679
150.0    0.157593
134.0    0.118451
110.0    0.104943
170.0    0.080587
Name: proportion, dtype: float64
```

In [27]:
```python
import matplotlib.pyplot as plt
import seaborn as sns

# Create a grouped dataframe for plotting
plot_data = data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])['TOT_SALES'].sum().rese

# Set plot size and style
plt.figure(figsize=(12, 8))
sns.barplot(data=plot_data, y='LIFESTAGE', x='TOT_SALES', hue='PREMIUM_CUSTOMER')

# Add labels and title
plt.title('Total Sales by Customer Segment')
plt.xlabel('Total Sales ($)')
plt.ylabel('Lifestage')
plt.legend(title='Premium Status', loc='center left', bbox_to_anchor=(1, 0.5))
plt.show()
```

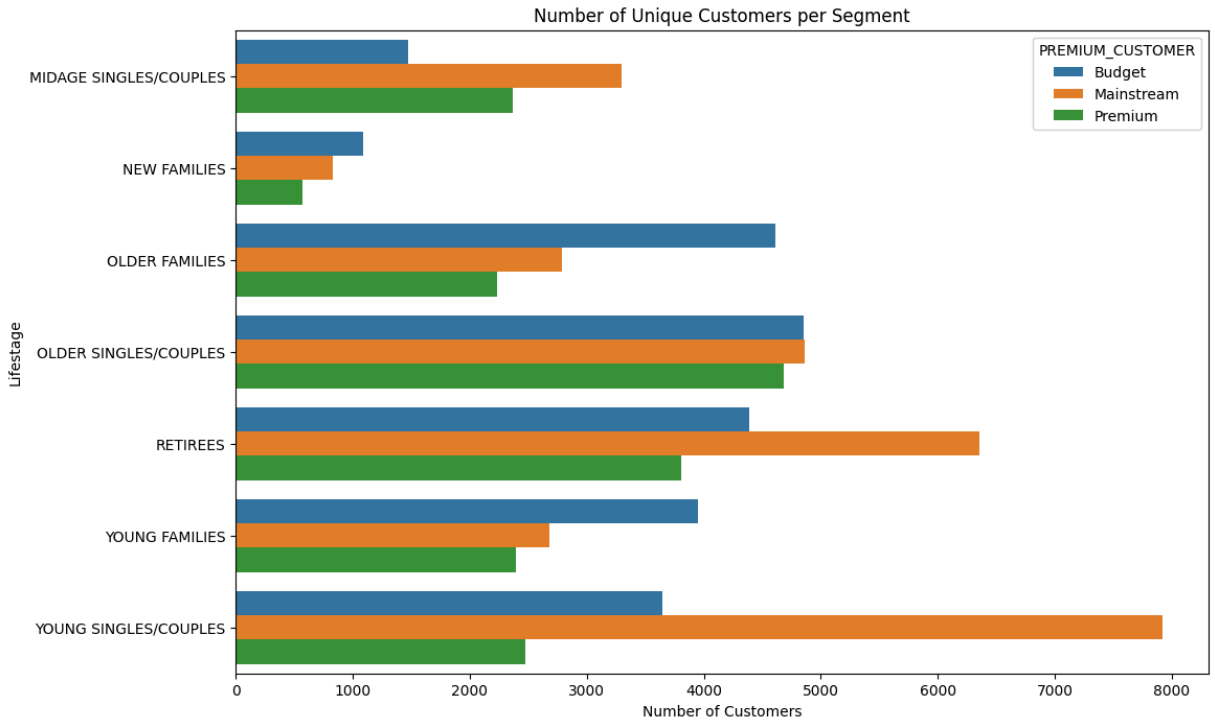


In [ ]:
```
# Total Sales by Segment (Bar Chart)
#This shows exactly where the money is coming from across those 21 segments.
```

In [28]:
```
# Group by segments to count unique loyalty cards
customer_counts = data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])['LYLTY_CARD_NBR']

plt.figure(figsize=(12, 8))
sns.barplot(data=customer_counts, y='LIFESTAGE', x='LYLTY_CARD_NBR', hue='PREMIUM_C

plt.title('Number of Unique Customers per Segment')
plt.xlabel('Number of Customers')
plt.ylabel('Lifestage')
plt.show()
```

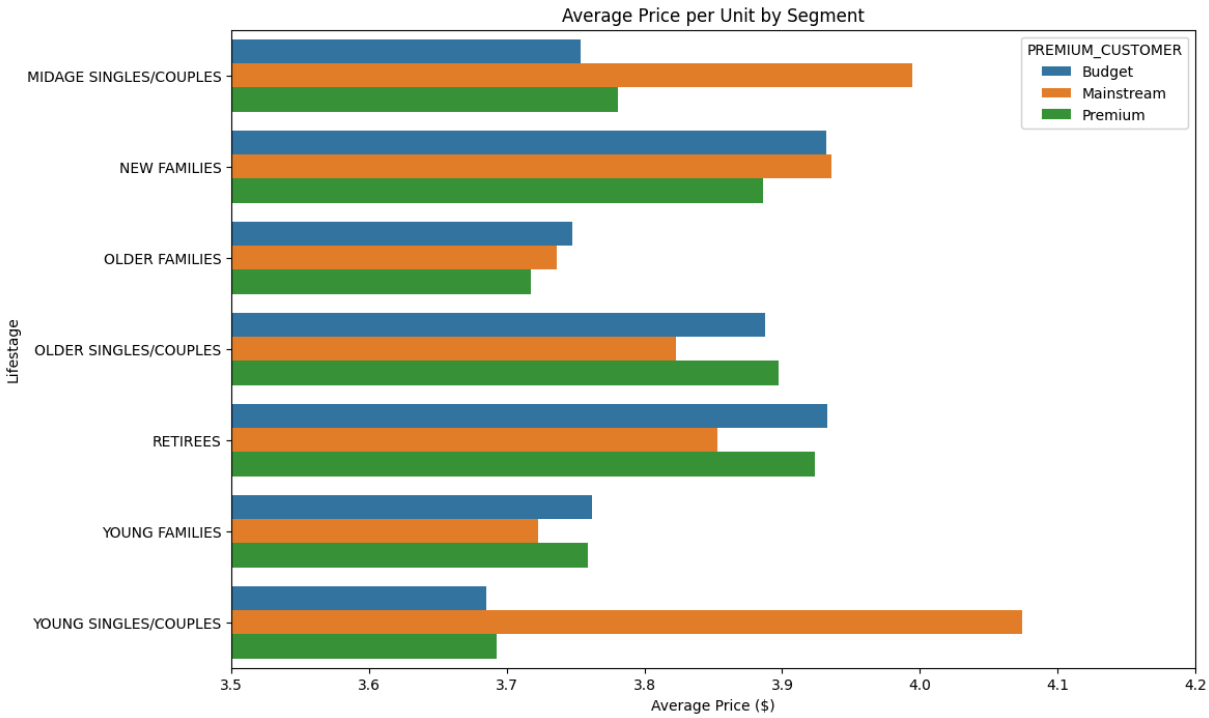


In [ ]:
```
# Number of Customers (To show the "Growth Opportunity")
# We found that Mainstream Young Singles have the most customers but aren't #1 in s
```

In [29]:
```
# Calculate average price: Total Sales / Total Quantity
avg_price = data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER']).apply(lambda x: x['TOT_
avg_price.columns = ['LIFESTAGE', 'PREMIUM_CUSTOMER', 'AVG_PRICE']

plt.figure(figsize=(12, 8))
sns.barplot(data=avg_price, y='LIFESTAGE', x='AVG_PRICE', hue='PREMIUM_CUSTOMER')

plt.title('Average Price per Unit by Segment')
plt.xlabel('Average Price ($)')
plt.ylabel('Lifestage')
plt.xlim(3.5, 4.2) # Zooming in to show the difference clearly
plt.show()
```



In [ ]:
```
#Average Price per Unit (The "Mainstream" Premium)
#This chart will visually prove that Mainstream Young Singles/Couples pay more for
```

In [32]:
```
from scipy.stats import ttest_ind

# Group 1: Mainstream Young Singles/Couples
mainstream_young = data[(data['LIFESTAGE'] == 'YOUNG SINGLES/COUPLES') & (data['PRE

# Group 2: Everyone else
others = data.loc[~((data['LIFESTAGE'] == 'YOUNG SINGLES/COUPLES') & (data['PREMIUM

# Perform t-test
t_stat, p_val = ttest_ind(mainstream_young, others.dropna())
print(f"P-value: {p_val}")
# If p-value < 0.05, the difference is statistically significant!
```

```
P-value: 1.419308440276165e-218
```

Executive Summary & Strategic Recommendations

Based on the data analysis of chip purchasing behavior across all customer segments, the following strategic recommendations are proposed for the upcoming category review:

1. Primary Target Segment: Mainstream Young Singles/Couples

- **The Insight:** While "Older Families - Budget" bring in the highest total revenue, **Mainstream Young Singles/Couples** represent the largest growth opportunity. They have the highest number of unique customers and the highest **Average Price per Unit ($4.07)**.

- **Statistical Significance:** A t-test confirmed a **p-value of 1.42e-218**, proving that this segment's preference for higher-priced chips is statistically significant and not due to random chance.

2. Product Optimization: Premium Brands & Specific Pack Sizes

- **The Insight:** This segment shows a clear preference for "Premium" flavored chips over traditional budget options. Their top brands are **Kettle** (19.7%), **Doritos**, and **Pringles**.
- **Pack Size Preference:** Sales are dominated by the **175g** size, followed by **134g** (the standard Pringles tube size).
- **Recommendation:** Focus the product mix on these specific brands and sizes to cater to the "Mainstream" taste profile.

3. Commercial Application & Placement

- **Targeting:** Implement marketing activations specifically for Mainstream Young Singles/Couples.
- **Placement:** Increase stock levels of Kettle and Pringles in stores located in high-density urban areas, near **major transit hubs**, and **commercial work centers** where young professionals shop.
- **Pricing Strategy:** Since this group is less price-sensitive, Julia can prioritize premium brand displays over deep-discount "Budget" brand promotions in these specific locations.